Lab 4 – Peripherals Programming on mbed Part 3: External Interrupt Control of the I²C TMP102 digital temperature module and Power Saving Analysis

Embedded Processors UESTC 2004 School of Engineering, University of Glasgow

Student Name: Chen YiChi

Student Number: [redacted]

UESTC 2004 Lab Assessment Criteria

Each individual lab will be evaluated using the below criteria (check the assessment table at the end as they :

- Demonstrating a functional embedded system

  This means each student will need to demonstrate a functional system for each task in the lab manual to the GTA and member of staff to get the full mark.

- Lab Questions (if any)
- Reflection and significance

  After the completion of programming and testing each of the "Tasks", you will be required to reflect, in a few lines, on your understanding of the tasks and its significance. This is a chance for you to ask yourselves "What you did?", "What it means?", and "What next?", meaning what is the significance of the undertaken task in the context of embedded systems and what you are learning in the lecture. Try to be innovative and think outside the box. Please, do not provide a summary of what you did as it will not be accepted.

- Adherence to coding good practice
  - Meaningful variable names – please avoid names like "x", "y", or personal names
  - Using block comments to explain what your code does at the beginning and using line comments to clarify what this line of code does.
  - Consistent indentation – Indentation helps in making the code readable and easy to interpret. See an example of what to do and what to avoid, below:

Good practice ✓

```
#include "mbed.h"
```

```
char switch_word ;         //the word we will send
char recd_val;            //the received value
int main() {
     async_port.baud(9600);      //set baud rate to 9600
     recd_val = 0;
     while (1)
     {
        switch_word=0xa0;
     }
}
```

Avoid this ✗

```
#include "mbed.h"
char switch_word ;         //the word we will send
char recd_val;            //the received value
int main() {
async_port.baud(9600);           //set baud rate to 9600
recd_val = 0;
while (1)
{
switch_word=0xa0;
}
}
```

All the best of luck.

The Embedded Processors Team,

Introduction

The TMP102 is a temperature sensor that communicates over I2C. In this lab, we will interface the TMP102 sensor with the NUCLEO L432KC mbed board using I2C and read the temperature data. We will perform several tasks to learn more about the sensor and how to use it.

Objective

The objective of the lab is to learn how to interface the TMP102 temperature sensor with the NUCLEO L432KC mbed board using I2C communication and to gain experience in developing embedded systems applications that use sensors and I2C communication. Through several tasks, such as reading and averaging temperature values, enabling shutdown and sleep modes, measuring current, and implementing external interrupts, students can develop skills in sensor interfacing, low power modes, and event-driven programming, which are essential in designing and building embedded systems applications. Additionally, the lab also aims to promote critical thinking and problem-solving skills by having students analyze and interpret sensor

readings and identify possible sources of error.

Please note that you need to create separate ".cpp" files for each task and put all together (compressed, e.g., ".zip") and upload them, with your lab exercise where you will have given answers to any questions and written your reflection, to the dedicated moodle link for this lab.

Requirements

Software:

- Keil arm online compiler (https://studio.keil.arm.com/auth/login/) – You will need to create an account to use the online compiler.

Hardware:

For this lab, the following components will be needed:

- NUCLEO L432KC mbed board
- TMP102 sensor
- Breadboard
- Jumper wires
- USB cable
- Multimeter
- Push button
- Pull Up/Pull Down Resistor

Lab Procedure:

Step 1: Setting up the hardware

- Connect the TMP102 sensor to the breadboard.
- Connect the power supply pins of the TMP102 sensor to the 3.3V pin of the NUCLEO L432KC mbed board.
- Connect the SDA and SCL pins of the TMP102 sensor to the corresponding I2C1 pins (D0 and D1) on the NUCLEO L432KC mbed board.
- Connect the ammeter to the power supply pin of the TMP102 sensor in the correct manner to measure the current **(show the ammeter connection to the GTA before powering the circuit).**
- Connect the push button to an external interrupt pin (A6) on the NUCLEO L432KC mbed board.
- Connect the NUCLEO L432KC mbed board to your computer using a USB cable.

Task 1: Reading Data from the TMP102 Sensors in Continuous Conversion Mode

1) Write the code to initialise the I²C (with slave address 0x90 – Check the datasheet if this doesn't work) interface and read the temperature data from the TMP102 sensor.
    - In your code configure the TMP102 once, outside of the while (1) loop.
2) Use the serial monitor to print the temperature data every 1 second.
3) Record 10 values on your lab book.
4) Cover the sensor with your hand and record the change in temperature when you get closer to the sensor and when you move further away from it.
5) Calculate the error in the readings by comparing the recorded temperature values with the actual temperature value. Note any sources of error, such as ambient temperature changes or sensor calibration issues.

[Answer in the below space]

```
#include "mbed.h"
#include <cstdio>

#define I2CSDA D0
#define I2CSDL D1


I2C TMP102(I2CSDA,I2CSDL);

const int TMP102Address = 0x48;// TMP102 address
char ConfigRegisterTMP102[3];//config register
char TemperatureRegister[2];//Temperanture Register
float Temperature;

void ConfigureTMP102()
{
    ConfigRegisterTMP102[0] = 0x01;
    ConfigRegisterTMP102[1] = 0x60;
    ConfigRegisterTMP102[2] = 0xA0;
    TMP102.write(TMP102Address<<1, ConfigRegisterTMP102,3);//write 3 bytes
}

// main() runs in its own thread in the OS
int main(){

    unsigned short M;
    char L;
    ConfigureTMP102();
    ConfigRegisterTMP102[0] = 0x00;//point to the tmp register
    TMP102.write(TMP102Address<<1, ConfigRegisterTMP102,1);
```

```
    printf("Initiating Temperature Reading..\n\r");

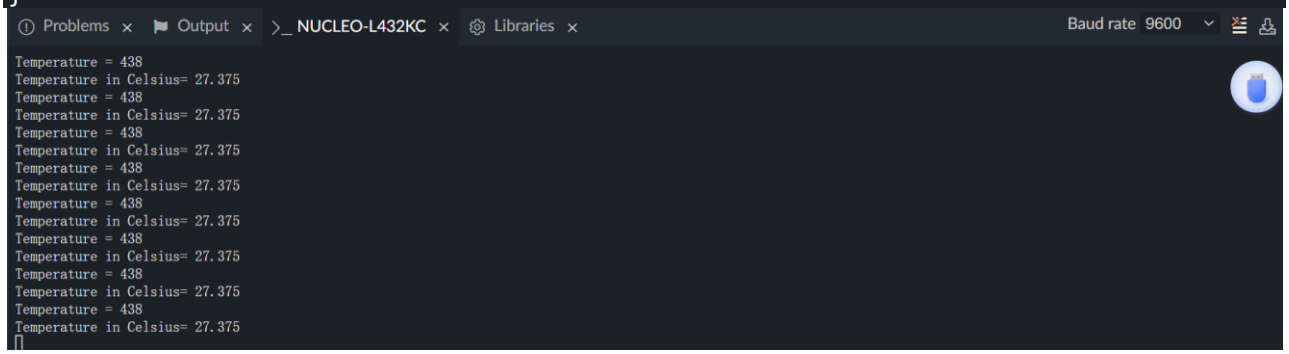
    while(1){
        TMP102.read(TMP102Address<<1, TemperatureRegister,2);//read 2 bytes

        M = TemperatureRegister[0]<<4;

        L = TemperatureRegister[1]>>4;

        M = M + L;
        Temperature = 0.0625*M;

        printf("Temperature = %u\n\r",M);
        printf("Temperature in Celsius= %.3f\n\r",Temperature);
        thread_sleep_for(500);
    }
}
```



Real temperature:26.9
Source of error : too close to my PC

Task 2: Error Reduction - Calculating Average Temperature from TMP102 Sensor

1) Modify the code to take 10 readings, one every second, from the TMP102 sensor and calculate the average temperature.
2) Print every temperature reading from the 10 collected followed by the average temperature on the serial terminal and take a screenshot into your lab book (example below).
3) Vary the temperature values by moving your hand close and away from the sensor and verify that the average is correct.

```
Temperature Reading number 0 = 22.437500
Temperature Reading number 1 = 22.437500
Temperature Reading number 2 = 22.437500
Temperature Reading number 3 = 22.437500
Temperature Reading number 4 = 22.437500
Temperature Reading number 5 = 23.187500
Temperature Reading number 6 = 23.500000
Temperature Reading number 7 = 22.937500
Temperature Reading number 8 = 22.625000
Temperature Reading number 9 = 22.625000
Average Temperature is = 22.706249
```

[Answer in the below space]

```
#include "mbed.h"
#include <cstdio>

#define I2CSDA D0
#define I2CSDL D1


I2C TMP102(I2CSDA,I2CSDL);

const int TMP102Address = 0x48;// TMP102 address
char ConfigRegisterTMP102[3];//config register
char TemperatureRegister[2];//Temperanture Register
const int n=10;
float Temperature[n];

void ConfigureTMP102()
{
    ConfigRegisterTMP102[0] = 0x01;
    ConfigRegisterTMP102[1] = 0x60;
    ConfigRegisterTMP102[2] = 0xA0;
    TMP102.write(TMP102Address<<1, ConfigRegisterTMP102,3);//write 3 bytes
}

// main() runs in its own thread in the OS
int main(){

    unsigned short M[10];
    char L;
    float Maverage;
    int i;
    ConfigureTMP102();
    ConfigRegisterTMP102[0] = 0x00;//point to the tmp register
    TMP102.write(TMP102Address<<1, ConfigRegisterTMP102,1);
    printf("Initiating Temperature Reading..\n\r");

    while(1){
        for (i=0;i<10;i++){
```

```
            TMP102.read(TMP102Address<<1, TemperatureRegister,2);//read 2 bytes

            M[i] = TemperatureRegister[0]<<4;

            L = TemperatureRegister[1]>>4;

            M[i] = M[i] + L;

            Temperature[i] = 0.0625*M[i];

            printf("Temperature %d = %.3f\n\r",i ,Temperature[i]);
            thread_sleep_for(500);
        }
        Maverage = (Temperature[0]+Temperature[1]+Temperature[2]+Temperature[3]+Te
mperature[4]+Temperature[5]+Temperature[6]+Temperature[7]+Temperature[8]+Temperatu
re[9])/10;
        printf("Average Temperature = %.3f\n\r",Maverage);

        thread_sleep_for(500);
    }
}
```

```
Temperature 0 = 25.875
Temperature 1 = 25.875
Temperature 2 = 25.875
Temperature 3 = 25.875
Temperature 4 = 25.875
Temperature 5 = 25.875
Temperature 6 = 25.875
Temperature 7 = 25.875
Temperature 8 = 25.875
Temperature 9 = 25.875
Average Temperature = 25.875
```

Task 3: TMP102 Shutdown (SD) Mode

1) Read the datasheet of the TMP102 sensor to understand how to enable the shutdown mode.
2) Modify the code in Task 1 to enable the shutdown mode.
3) Verify that the sensor is in shutdown mode by measuring the current with the ammeter and recording it in your lab book.

[Answer in the below space]

```
#include "mbed.h"
#include <cstdio>
```

```
#define I2CSDA D0
#define I2CSDL D1


I2C TMP102(I2CSDA,I2CSDL);

const int TMP102Address = 0x48;// TMP102 address
char ConfigRegisterTMP102[3];//config register
char TemperatureRegister[2];//Temperanture Register
float Temperature;

void ConfigureTMP102()
{
    ConfigRegisterTMP102[0] = 0x01;
    ConfigRegisterTMP102[1] = 0x61;
    ConfigRegisterTMP102[2] = 0xA0;
    TMP102.write(TMP102Address<<1, ConfigRegisterTMP102,3);//write 3 bytes
}

// main() runs in its own thread in the OS
int main(){

    unsigned short M;
    char L;
    ConfigureTMP102();
    ConfigRegisterTMP102[0] = 0x00;//point to the tmp register
    TMP102.write(TMP102Address<<1, ConfigRegisterTMP102,1);
    printf("Initiating Temperature Reading..\n\r");

    while(1){
        TMP102.read(TMP102Address<<1, TemperatureRegister,2);//read 2 bytes

        M = TemperatureRegister[0]<<4;

        L = TemperatureRegister[1]>>4;

        M = M + L;
        Temperature = 0.0625*M;

        printf("Temperature = %u\n\r",M);
        printf("Temperature in Celsius= %.3f\n\r",Temperature);
        thread_sleep_for(500);
    }
}
```

```
Temperature 0 = 25.938
Temperature 1 = 25.938
Temperature 2 = 25.938
Temperature 3 = 25.938
Temperature 4 = 25.938
Temperature 5 = 25.938
Temperature 6 = 25.938
Temperature 7 = 25.938
Temperature 8 = 25.938
Temperature 9 = 25.938
Average Temperature = 25.938
```

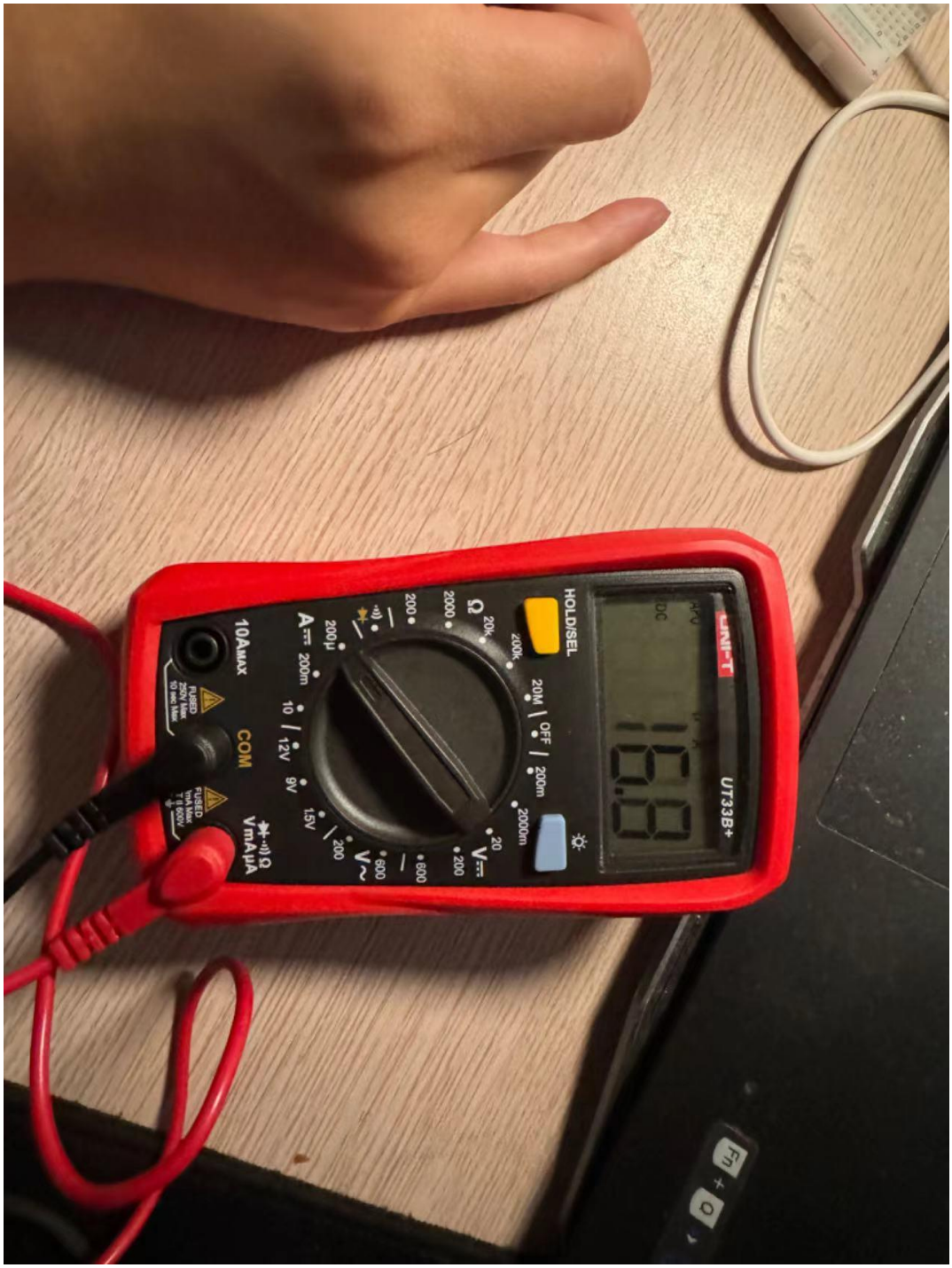
Temperature value are same because of shut down mode.



Task 4: Current Measurement With and Without SD Mode Enabled

1) Repeat Task 3 and measure the current flowing through the TMP102 sensor using the ammeter with and without shutdown mode enabled.
2) Record the current values in your lab book and compare the values to determine the energy savings of using shutdown mode.

[Answer in the below space]



Task 5: Trigger TMP102 to Measure the Temperature Using an External Interrupt from a Push Button

1) Modify the code in **Task 1** to implement an external interrupt from a push button, on a rising edge, that triggers the TMP102 to take a reading and display them on a serial monitor, with SD mode enabled.
2) Push the button 10 times to take 10 readings and record them in your lab book.
6) Repeat this last step but cover the sensor with your hand and record the change in temperature when you get closer to the sensor and when you move further away from it.
    - You will notice that the temperature values do not change. Identify why?
    - **Bonus Task (10%):** amend the code to record the changes in temperature every time you press the button.

[Answer in the below space]

```
#include "mbed.h"


#define I2CSDA D0
#define I2CSDL D1

DigitalIn switch_ip1(D5, PullDown);

I2C TMP102(I2CSDA,I2CSDL);

const int TMP102Address = 0x90;// TMP102 address
char ConfigRegisterTMP102[3];//config register
char TemperatureRegister[2];//Temperanture Register
float Temperature;

void ConfigureTMP102()
{
    ConfigRegisterTMP102[0] = 0x01;
    ConfigRegisterTMP102[1] = 0x60;
    ConfigRegisterTMP102[2] = 0xA0;
    TMP102.write(TMP102Address, ConfigRegisterTMP102,3);//write 3 bytes
}

// main() runs in its own thread in the OS
int main(){

    unsigned short M;
    char L;
    ConfigureTMP102();
    ConfigRegisterTMP102[0] = 0x00;
    TMP102.write(TMP102Address, ConfigRegisterTMP102,1);
```

```
    printf("Initiating temperature..\n\r");

    while(1){
        float sum=0;

        if (switch_ip1 == 1)
        {
            ThisThread::sleep_for(20ms);    //按键防抖
            if (switch_ip1 == 1)
            {
                TMP102.read(TMP102Address, TemperatureRegister,2);//read 2 bytes
                M = TemperatureRegister[0]<<4;
                L = TemperatureRegister[1]>>4;
                M = M + L;
                Temperature = 0.0625*M;
                printf("Temperature = %.3f\n\r",Temperature);
            }
            ThisThread::sleep_for(2000ms);    //按键防抖
        }


    }
}
```

```
Temperature = 25.250
Temperature = 25.313
Temperature = 25.313
Temperature = 25.500
Temperature = 25.938
Temperature = 26.563
Temperature = 27.000
Temperature = 27.438
Temperature = 27.688
Temperature = 28.000
```

Task 6: Reflection

Reflect on your understanding and significance of the undertaken tasks in the context of embedded systems and what you have learned. Consider questions such as "**What did you learn?**", "**What does it mean?**", and "**What's next?**". This is an opportunity to think innovatively and critically about the importance of the tasks completed. Avoid merely summarizing what you did and instead consider the broader implications and potential applications of your newfound knowledge in the field of embedded systems.

[Answer in the below space]

In the next

Lab Assessment

Marking Criteria	Student's Score
Task 1 Demonstration (5%)	
Task 2 Demonstration (10%)	
Task 3 Demonstration (15%)	
Task 4 Demonstration (5%)	
Task 5 Demonstration (30%)	
Reflection (20%)	
Adherence to coding good practice (15%)	
Bonus Task 5 (10%)	